

Task 01

1. Write a program. Allocate an int using mmap with MAP_PRIVATE | MAP_ANONYMOUS. Set it to 10. Call fork(). In the child, set it to 99 and exit. In the parent, call wait(), then print the value. What do you see?
2. Change MAP_PRIVATE to MAP_SHARED. Run again. What do you see now?
3. Write one line in a comment explaining why the result changed.

Task 02

Write a program, allocate an int *counter via mmap(MAP_SHARED | MAP_ANONYMOUS). Initialize to 0.

- Fork child 1. Child 1 increments counter by 5 (one by one, with a printf each time) then exits.
- Fork child 2. Child 2 also increments counter by 5 then exits.
- Parent waits for both children using waitpid, then prints the final counter value.
- Call munmap() at the end.

Task 03

Define a shared header that declares a structure with an integer field and a character array (size 32), and define a constant name for the shared memory object.

- Write a program `writer.c` that creates the shared memory object, sets its size, maps it into memory with shared access, assigns values to the structure fields, unmaps and closes the memory, and prints a completion message. Do not remove the shared memory object.
- Write a program `reader.c` that opens the existing shared memory object, maps it into memory, reads and prints the stored values, then unmaps, closes, and removes the shared memory object. Compile both programs and execute them by running the writer first and then the reader.

Task 04

Define a shared structure containing two integer fields (`counter` and `done`) and declare a constant name for the shared memory object.

- Write a program `producer.c` that creates and maps the shared memory, initializes the fields, repeatedly updates the counter with a delay while printing its value, sets a completion flag after finishing, and performs cleanup without removing the shared memory object.
- Write a program `consumer.c` that opens and maps the same shared memory, continuously reads and prints the counter value until the completion flag is set, then prints a final message with the last counter value and performs full cleanup including removing the shared memory object. Open two terminals and execute both programs concurrently, running the producer in one terminal and the consumer in the other.